من العشري إلى الثنائي، ووحدات القياس

الدرس 2 (تابع) — SST-ICT-01، 1IOT — القسمة المتتالية، bit/byte/word

16/09/2026

1 من العشري إلى الثنائي: طريقة القسمة المتتالية

في الدرس السابق حولنا أعدادًا ثنائية إلى عشرية. لنرَ الآن **الطريق** **المعاكس**.

تعريف

لتحويل عدد عشري إلى ثنائي **نقسم** **بشكل** **متكرر** **على** **2**، ونسجل في كل مرة **الباقي** (0 أو 1، **resto**)، حتى نصل إلى **خارج** **قسمة** (**quoziente**) **يساوي** **0**. يُقرأ العدد الثنائي **من** **آخر** **باقٍ** **محسوب** **إلى** **الأول** (**من** **الأسفل** **إلى** **الأعلى**).

مثال

تحويل 39 إلى الثنائي:

	الباقي	خارج القسمة	القسمة
(آخر بت)	1	19	$39 \div 2$
	1	9	$19 \div 2$
	1	4	$9 \div 2$
	0	2	$4 \div 2$
	0	1	$2 \div 2$
(أول بت)	1	0	$1 \div 2$

بقراءة البواقي **من** **الأسفل** **إلى** **الأعلى**: 100111_2.

التحقق (كما رأينا في الدرس السابق): $39 = 1 + 2 + 4 + 0 + 0 + 32 = 100111_2$. صحيح!

ملاحظة

تحقق **دائمًا** **من** **النتيجة**: بعد تحويل عدد، **أعد** **تحويله** في الاتجاه المعاكس لتتأكد من العودة إلى العدد الأصلي. هذه أبسط طريقة لاكتشاف **خطأ** **في** **الحساب** – وقد حدث ذلك في القسم أيضًا، في مثال ثانٍ بعدد أكبر (حوالي 500-600): ضاع رقم أثناء الحساب، وظهر الخطأ عند مقارنة المجموع بالقيمة الأصلية.

مثال

مثال ثالث للتدريب: تحويل **192** إلى الثنائي.

القسمة	خارج القسمة	الباقي	
$192 \div 2$	96	0	
$96 \div 2$	48	0	
$48 \div 2$	24	0	
$24 \div 2$	12	0	
$12 \div 2$	6	0	
$6 \div 2$	3	0	
$3 \div 2$	1	1	
$1 \div 2$	0	1	(أول بت)

بقراءة البواقي **من** **الأسفل** **إلى** **الأعلى**: 11000000_2 – البتان **الأكثر** **أهمية** فقط قيمتهما 1، وكل البقية 0.

2 Bit و byte و word: أسماء الأشياء

تعريف

البت (BInary digiT، "رقم ثنائي") هو **أصغر** **معلومة** يمكن للحاسوب التعامل معها: 0 أو 1. لا يوجد شيء أصغر – البت هو "**ذرة**" المعلومات الرقمية.

مثال

البايت الذي **كل** **بتاته** **1** (11111111_2) يقابل العدد العشري **255** – وهو **أقصى** **قيمة** يمكن تمثيلها بـ8 بتات، بما يتوافق مع $2^8 = 256$ تركيبة ممكنة (من 0 إلى 255).

تعريف

مجموعة من **8** **بتات** تسمى **بايت** (**byte**). مجموعة من **2** **بايت** (16 بت) تسمى **word**. أما المجموعات الأكبر فيُذكر عادةً **عدد** **البتات** مباشرة (مثل "سجل من 32 بت").

ملاحظة

عمليًا، نادرًا ما نتعامل مع بت واحد منفرد: نعمل غالبًا **على** **مستوى** **البايت**. لذلك فإن **وحدة** **القياس** المرجعية لنا هي **البايت**، مع أن البت يبقى أصغر وحدة من حيث المفهوم.

2.1 كم "صندوقًا" نحتاج؟ أحجام المتغيرات

يمكن أن تكون **للمتغيرات** (**variabili**، أي "**أدراج**" الذاكرة التي تحتوي بياناتنا) أحجام مختلفة، دائمًا **من** **مضاعفات** **البايت**: 1 بايت (8 بتات، حتى 255)، 2 بايت (16 بت، حتى 65.535)، 4 بايت (32 بت، حتى نحو 4,3 مليار)، حتى 8 بايت (64 بت) في الحواسيب الحديثة.

تنبيه

اختيار **متغير** **أكبر** من اللازم **ليس** **خطأ**، لكنه **يهدر** **الذاكرة**. إذا كانت قيمة معلومة لا تتجاوز 15 (مثل عدد الطلاب في صف)، فلا حاجة إلى متغير من 32 بت: 8 بتات أكثر من كافية. **تحسين** حجم المتغيرات **ممارسة** **جيدة**، وليس تفصيلًا صغيرًا.

3 المتغيرات: "صناديق" الذاكرة

لننتقل الآن إلى الحديث عن **البرمجة**. لكي نبرمج علينا أن نتعامل مع البتات، لكن خصوصًا مع **البايتات**. البرنامج هو **قائمة** **تعليمات** **مرتبة** – لكن ماذا يفعل الحاسوب فعليًا؟ في الواقع يعرف الحاسوب **شيئًا** **أساسيًا** **واحدًا**: **معالجة** **المعلومات** (البتات). كل ما عدا ذلك (اللعب، الحساب، عرض الصور) مبني بتركيب هذه القدرة الأساسية الواحدة على مستويات متزايدة التعقيد.

إذا أردنا إجراء حسابات فإننا نحتاج إلى **الأعداد** – وقد رأينا أن الحاسوب يمثلها من الداخل بالشكل **الثنائي**. لكن أين "توضع" هذه الأعداد فعليًا في الحاسوب؟ البرمجة تعني، من بين أمور أخرى، تعريف وتحديد "**صناديق**" افتراضية داخل الحاسوب نضع فيها الأعداد.

ملاحظة

تشبيه الأدراج: فكر في الخزانة التي في غرفتك، وفيها عدة **أدراج** – واحد للجوارب، وواحد للقمصان، وواحد للملابس الداخلية، وواحد للسراويل. كل درج يحتوي عددًا من الأشياء وله اسم (مثل "درج الجوارب"، وفيه ربما 12 جوربًا). هذه الحاويات تشبه الحاويات التي يحفظ فيها الحاسوب **البيانات** والمعلومات والأعداد المستخدمة في الحسابات.

تعريف

المصطلح **التقني** لهذه "الأدراج" هو **المتغير** (**variabile**): المتغير هو **درج** **في** **ذاكرة** الحاسوب يمكننا أن نضع فيه معلومات – وهذه المعلومات **دائمًا** **أعداد** **فقط**. لا يمكننا أن نضع قطعة ملابس حقيقية في درج افتراضي: يمكننا فقط أن نضع **عدد** القطع التي لدينا (مثل 3، 7، 42).

مثال

يمكن أن تكون للصناديق **أحجام** **مختلفة**. الصندوق **من** **1** **بايت** يحتوي أعدادًا **من** **0** **إلى** **255** ($2^8 = 256$ تركيبة). وهو مناسب مثلًا لتمثيل عدد أزواج السراويل (لا أحد يملك أكثر من 255)، لكنه **صغير** **جدًا** لتمثيل عدد حبات الأرز في طبق ريزوتو مثلًا (255 حبة ستكون حصة مضحكة).

نحتاج إذن إلى **صندوق** **أكبر**: **من** **2** **بايت** **(16)** **بت**). انتبه: مضاعفة حجم الصندوق **لا** **تضاعف** **أقصى** **عدد** يمكن تمثيله – فبـ16 بت نصل إلى 2^{16} = **65.536** **تركيبة** **ممكنة** (من 0 إلى 65.535)، وهي **قفزة** **أكبر** **بكثير** من مجرد الضعف.

وللأعداد الأكبر ننتقل إلى صندوق **من** **4** **بايت** **(32)** **بت)**: 2^{32} يعطي نحو 4 مليارات و300 مليون (4.294.967.296) تركيبة ممكنة. ويمكن أن تصل الصناديق إلى **8** **بايت** **(64)** **بت)**: نحو $1{,}8 \times 10^{19}$ تركيبة، وهو عدد هائل للاستخدامات العادية.

ملاحظة

حاسوبك المنزلي اليوم ينشئ عادةً **متغيرات من 64 بت**: ونادرًا ما يعمل بصناديق من 1 أو 2 بايت في العمليات الداخلية الأساسية. تاريخيًا **ازداد الحجم القياسي مع الزمن**: قبل 15-20 سنة كانت الحواسيب تعمل **بـ32 بت**، وقبل 40 سنة بـ16 بت، وقبل 50-60 سنة بـ8 بتات.

4 وحدات القياس: kilo و mega و giga و tera...

تعريف

المضاعفات القياسية المستخدمة لقياس الكميات الكبيرة من البايتات تتبع تسلسلًا من **قوى العدد 1000** (10^3): **kilo** (10^3)، **mega** (10^6)، **giga** (10^9)، **tera** (10^{12})، **peta** (10^{15})، **exa** (10^{18})، **zetta** (10^{21})، **yotta** (10^{24})، وفي التسمية الأحدث **ronna** (10^{27}) وما بعدها. للتحقق من الأسماء الدقيقة للبادئات القصوى، فإن أيسر مصدر هو صفحة Wikipedia الخاصة **ببادئات القياس**.

البادئة	الرمز	العدد كاملًا	قوة العدد 10
kilo	K	1.000	10^3
mega	M	1.000.000	10^6
giga	G	1.000.000.000	10^9
tera	T	1.000.000.000.000	10^{12}
peta	P	1.000.000.000.000.000	10^{15}
exa	E	1.000.000.000.000.000.000	10^{18}
zetta	Z	1.000.000.000.000.000.000.000	10^{21}
yotta	Y	1.000.000.000.000.000.000.000.000	10^{24}

مثال

القرص الذي سعته 4 **terabyte** (TB) يحتوي نحو 4×10^{12} بايت، أي نحو 32 ألف مليار بت ($= 4 \times 8 \times 10^{12}$). يجب أن تكون قيمة كل بت صحيحة كي يعمل الحاسوب: إنه نظام يتطلب **دقة تصنيع** شبه مطلقة.

ملاحظة

يُستخدم منطق البادئات نفسه "بالعكس" أيضًا، **للكميات الصغيرة جدًا**: **milli** (10^{-3}، جزء من ألف)، **micro** (10^{-6})، **nano** (10^{-9})، **pico** (10^{-12}). فالنانومتر (**nanometro**) مثلًا من رتبة حجم الذرات.

البادئة	الرمز	العدد كاملًا	قوة العدد 10
milli	m	0,001	10^{-3}
micro	μ	0,000001	10^{-6}
nano	n	0,000000001	10^{-9}
pico	p	0,000000000001	10^{-12}

5 ما يجب تذكره من هذا الدرس

- لتحويل **عدد عشري إلى ثنائي**: **قسمة متتالية على 2**، مع قراءة **البواقي** من الأسفل إلى الأعلى.

- **البت** (BInary digiT) هو أصغر معلومة رقمية (0/1)؛ **البايت** مجموعة من 8 بتات؛ **word** مجموعة من 2 بايت.
- **المتغير** هو "درج" في الذاكرة نضع فيه أعدادًا (**ولا شيء آخر أبدًا**): **أحجام المتغيرات من مضاعفات البايت** (1، 2، 4، 8 بايت)، واختيار **الأصغر المناسب** يمنع **هدر الذاكرة**.
- **المضاعفات** (kilo و mega و giga و tera...) تتقدم **بقوى العدد 1000** (10^3 في كل قفزة).
- **تحقق** دائمًا من حساب التحويل **بإعادة التحويل** في الاتجاه المعاكس.

مثال

للتحضير للدرس القادم: حوّل العدد 100 إلى الثنائي بطريقة **القسمة المتتالية**، ثم تحقق من النتيجة بإعادته إلى العشري.

6 مسرد: كلمات إيطالية تُستخدم في القسم

- **divisione**: قسمة
- **quoziente / resto**: خارج القسمة / الباقي
- **dal basso verso l'alto**: من الأسفل إلى الأعلى
- **verifica / verificare**: تحقق / يتحقق (و**verifica** تعني أيضًا الاختبار)
- **variabile**: متغير
- **cassetto / scatola**: درج / صندوق
- **memoria**: ذاكرة
- **dati**: بيانات
- **prefisso / simbolo**: بادئة / رمز
- **miliardi**: مليارات (1.000.000.000)